



Faza 2 - Model podataka i perzistencije

Arhitektura i projektovanje softvera

Kanbify

Članovi tima:

Aleksandar Gospavić 19095

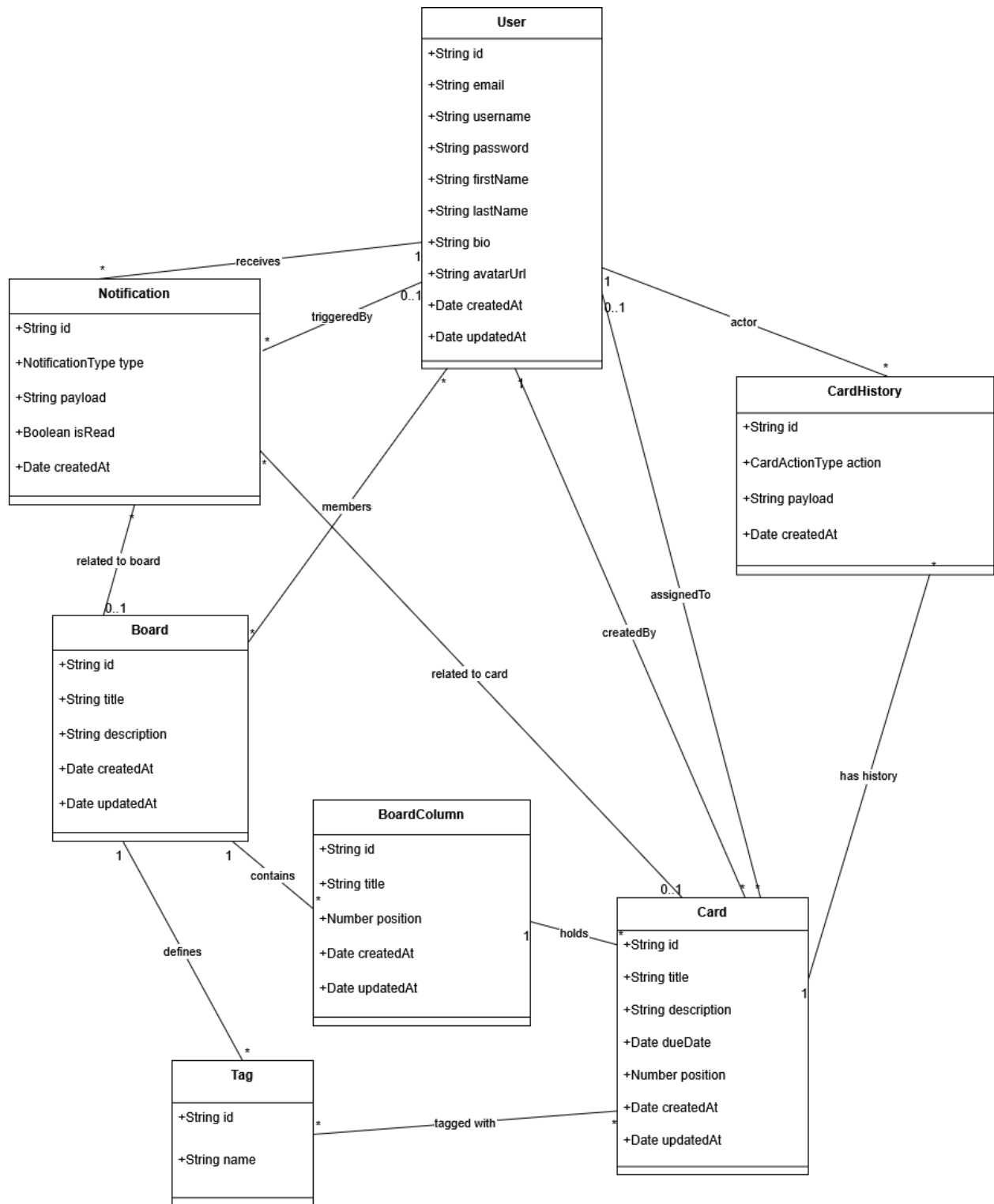
David Marković 19211

Sadržaj

1. Model podataka.....	2
Dijagram klasa.....	2
User.....	3
Board.....	3
BoardColumn.....	3
Card.....	3
Tag.....	3
CardHistory.....	3
Notification.....	3
2. Model perzistencije.....	4
Dijagram entiteta.....	4
User Entitet.....	5
Board Entitet.....	5
BoardMember Entitet.....	5
BoardColumn Entitet.....	5
Card Entitet.....	5
Tag i CardTag Entiteti.....	5
CardHistory Entitet.....	5
Notification Entitet.....	6

1. Model podataka

Dijagram klasa



User

Klasa koja apstrahuje krajnjeg korisnika sistema. Pored osnovnih identifikacionih podataka, ona objedinjuje profilne informacije (biografija, avatar) i služi kao centralna tačka za povezivanje sa svim akcijama u sistemu (kreiranje kartica, članstvo na tablama, primanje notifikacija).

Board

Predstavlja radnu površinu ili projekat. Ovo je korenski objekat agregacije koji logički grupiše kolone i kartice. Odgovoran je za definisanje konteksta rada, uključujući naslov i opis projekta, kao i upravljanje članovima koji imaju pristup tom sadržaju.

BoardColumn

Definiše vertikalnu podelu table (npr. "To Do", "In Progress", "Done"). Njena glavna uloga je strukturna organizacija toka posla (workflow). Sadrži informaciju o svojoj poziciji kako bi se obezbedio tačan redosled prikazivanja faza rada.

Card

Centralni element domena koji predstavlja pojedinačni zadatak ili jedinicu posla. Sadrži sve bitne informacije za izvršenje zadatka: rokove, opis i trenutnu poziciju. Dizajnirana je da bude dinamična – kreće se kroz kolone i menja stanja tokom životnog ciklusa.

Tag

Pomoćna klasa za kategorizaciju. Omogućava vizuelno označavanje i filtriranje kartica unutar jedne table. Njena uloga je da pruži dodatni sloj semantike zadacima (npr. "Bug", "Feature", "Urgent").

CardHistory

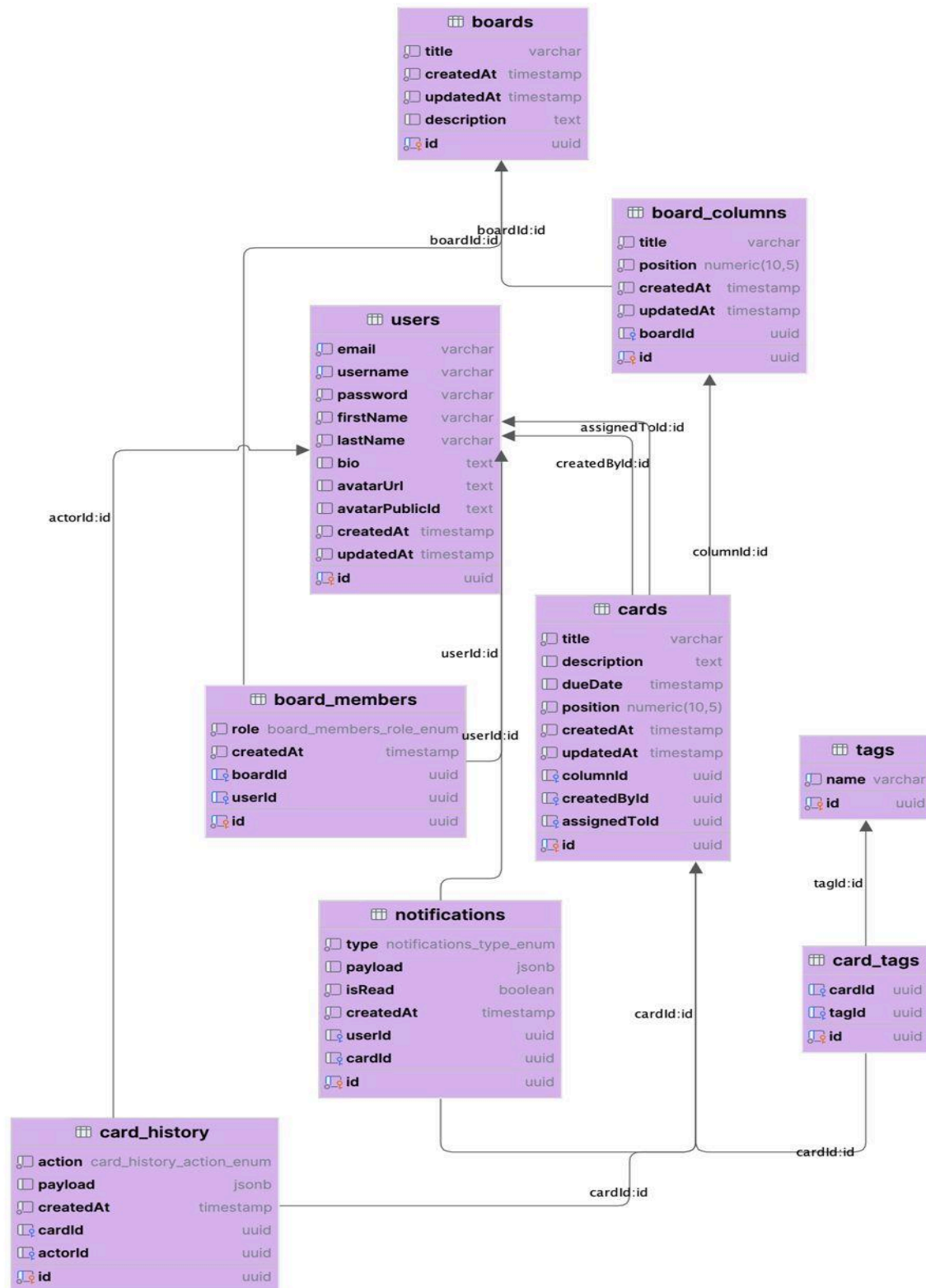
Klasa zadužena za reviziju i praćenje promena (audit log). Njena svrha je da trajno beleži svaku ključnu aktivnost nad karticom (pomeranje, izmena, dodela), čuvajući informaciju o tome ko je uradio akciju i šta se tačno promenilo.

Notification

Mehanizam za asinhrono obaveštavanje korisnika. Ova klasa modeluje systemske alerte koji nastaju kao reakcija na događaje (npr. kada je korisnik dodeljen na karticu), povezujući inicijatora događaja sa primaocem obaveštenja.

2. Model perzistencije

Dijagram entiteta



User Entitet

Mapira se na tabelu users. Pored standardnih polja, koristi **@Exclude** dekorator za polje password radi bezbednosti. Definiše **OneToMany** relacije ka članstvima na tablama i notifikacijama, omogućavajući kaskadno brisanje povezanih podataka.

Board Entitet

Perzistira podatke o tablama u tabeli boards. Koristi **UUID** kao primarni ključ. Sadrži relacije ka **BoardColumn** i **Tag** entitetima, kao i vezu ka **BoardMember** tabeli koja razrešava **Many-to-Many** odnos sa korisnicima.

BoardMember Entitet

Predstavlja asocijativnu (veznu) tabelu **board_members** sa kompozitnim jedinstvenim ograničenjem (board, user). Pored stranih ključeva, čuva i atribut role (Enum: **ADMIN**, **MEMBER**), definišući nivo privilegija korisnika na konkretnoj tabli.

BoardColumn Entitet

Čuva se u tabeli **board_columns**. Koristi numeric tip podatka za polje position kako bi se omogućilo precizno i fleksibilno rangiranje kolona bez potrebe za reindeksiranjem celog niza prilikom pomeranja.

Card Entitet

Najsloženiji entitet u tabeli cards. Pored stranih ključeva ka koloni i korisnicima (kreator, zaduženi), definiše kaskadne operacije za tagove. Čuva dueDate kao timestamp i poziciju kao decimalni broj radi lakšeg sortiranja unutar kolone.

Tag i CardTag Entiteti

Tag entitet (tabela **tags**) je jedinstven na nivou table (Unique constraint). Relacija između kartica i tagova je realizovana preko veznog entiteta **CardTag** (tabela **card_tags**), što omogućava efikasno upravljanje **Many-to-Many** vezom i optimizovane upite.

CardHistory Entitet

Skladišti istoriju u tabeli **card_history**. Koristi jsonb tip kolone za payload atribut, što omogućava fleksibilno čuvanje različitih struktura podataka (npr. stara i nova vrednost) u zavisnosti od tipa akcije (CardActionType Enum), bez potrebe za menjanjem šeme baze.

Notification Entitet

Nalazi se u tabeli notifications. Dizajniran je da bude polimorfan u logičkom smislu – sadrži opcione strane ključeve ka Card i Board entitetima, zavisno od konteksta notifikacije. Takođe koristi jsonb za dodatne podatke i Enum za tip notifikacije.

3. Mehanizam mapiranja i implementacija

Za sloj perzistencije i komunikaciju sa bazom podataka odabran je TypeORM okvir unutar NestJS ekosistema, uz primenu Code-First pristupa. Ovakva arhitektura omogućava da se kompletna šema baze podataka (tabele, kolone, strani ključevi) generiše i automatski ažurira direktno na osnovu definisanih TypeScript klasa (entiteta), čime se osigurava potpuna sinhronizacija između aplikativnog modela i same baze.

Implementacija se oslanja na moderne mehanizme mapiranja, uključujući automatsko generisanje UUID identifikatora radi globalne jedinstvenosti zapisa i upravljanje vremenskim žigovima (createdAt, updatedAt) bez potrebe za ručnim ažuriranjem. Posebna prednost ovog rešenja je korišćenje JSONB tipova podataka za fleksibilno čuvanje kompleksnih struktura (poput istorije izmena i notifikacija), čime je postignut optimalan balans između strogog integriteta relacione baze i dinamičnosti koju zahteva moderna web aplikacija.